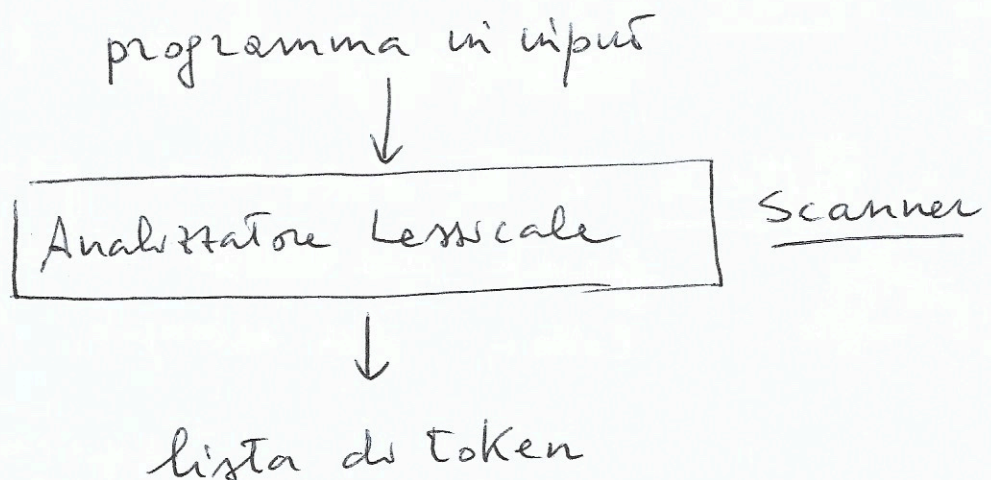


# Capitolo 3

## Analisi Lessicale - Linguaggi Regolari

Analisi Lessicale: riconoscere nella stringa in ingresso gruppi/sequenze di simboli che corrispondono a specifiche categorie sintattiche (ad es: identificatori, parole riservate, operatori aritmetici, ...)

La stringa in input è trasformata in una sequenza di simboli astratti, detti token



# Cos'è un token?

(2)

Token = coppia (nome, valore)

- nome = simbolo astratto che rappresenta una categoria sintattica
- valore = una sequenza di simboli del testo in ingresso

ad es.:  $\langle \text{Ide}, x_1 \rangle$

Dato il token  $\langle \text{Ide}, x_1 \rangle$ , diciamo che

- Nome (Ide) è l'informazione che identifica una classe di token (identificatori)
- Valore ( $x_1$ ) è l'informazione che identifica uno specifico token (l'identificatore  $x_1$ )
- Pattern è la descrizione generale della forma dei valori di una classe di token. Ad esempio,  $(x|y)(x|y|o|1)^*$   
espressione regolare  
(che vedremo)
- Lessema è una stringa istanza di un pattern.  
Nel nostro esempio  $x_1$  è un lessema istanza del pattern  $(x|y)(x|y|o|1)^*$



Vedremo che ad ogni "nome" di categoria<sup>(3)</sup> sintattica è associato un "pattern" che specifica i possibili "valori" che possono essere presi per quel nome, come "lessemi".

---

### Esempio

Dalla stringa C

```
if (x == 0) printf("zero")
```

un analizzatore lessicale potrebbe produrre la seguente sequenza di token

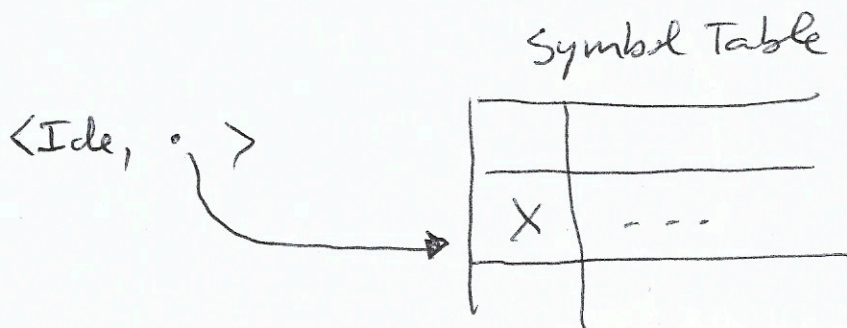
<IF> <(> <Ide, x> <OPREL, ==>

<CONST-NUM, 0> <)> <Ide, printf>

<(> <CONST-STRING, zero> <)>

In realtà, normalmente lo scanner associa agli identificatori un indirizzo nella tabella dei simboli

<Ide, x> è in realtà <Ide, puntatore alla Tabella dei simboli>



# Espressioni Regolari

(4)

Fissato un alfabeto  $A = \{a_1, a_2, \dots, a_n\}$ ,  
definiamo le espressioni regolari su  $A$  con la  
seguente BNF

$$r ::= \emptyset \mid \epsilon \mid a \mid r \cdot r \mid r \mid r \mid r^*$$

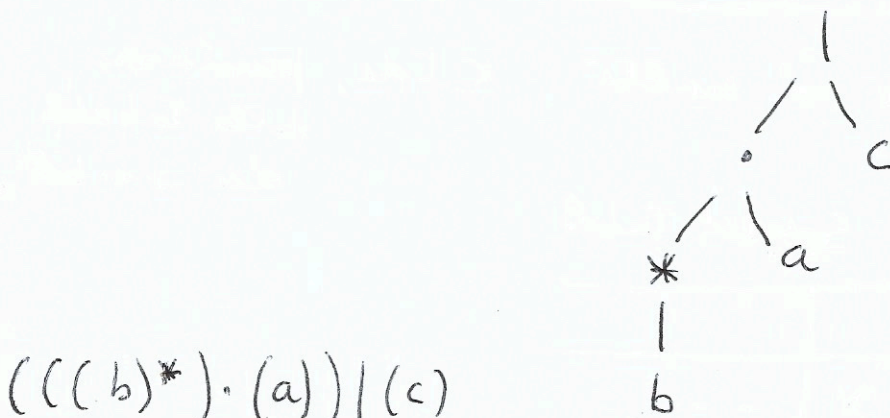
$\uparrow$   
 $\forall a \in A$

(Questa è una sintassi astratta ambigua, per  
disambiguare, possiamo usare le parentesi)

Per semplicità, si assume che:

- la concatenazione, la disgiunzione  
(e anche la ripetizione) associano a sx
- la precedenza tra gli operatori  
Sia:  $* > \cdot > |$
- la concatenazione  $\cdot$  è di solito  
omessa

per cui, ad esempio,  $b^* a \mid c$  corrisponde  
all'albero sintattico



$((b)^* \cdot (a)) \mid (c)$

secondo la sintassi del libro



# Linguaggio denotato da una espressione regolare (5)

Dato l'alfabeto  $A$ , definiamo la funzione

$$\mathcal{L}: \text{Exp-Reg} \rightarrow \mathcal{P}(A^*)$$

Come segue:

simboli di espressione regolare

$$\mathcal{L}[\emptyset] = \emptyset \leftarrow \text{linguaggio vuoto}$$

$$\mathcal{L}[\epsilon] = \{\epsilon\} \leftarrow \text{ling. che contiene una sola stringa, quella vuota}$$

$$\mathcal{L}[a] = \{a\}$$

$$\mathcal{L}[r_1 \cdot r_2] = \mathcal{L}[r_1] \cdot \mathcal{L}[r_2]$$

$$\mathcal{L}[r_1 | r_2] = \mathcal{L}[r_1] \cup \mathcal{L}[r_2]$$

$$\mathcal{L}[r^*] = (\mathcal{L}[r])^*$$

Se usate le parentesi:  $\mathcal{L}[(r)] = \mathcal{L}[r]$

Ricorda che:

$$L_1 \cdot L_2 = \{xy \mid x \in L_1, y \in L_2\}$$

$$L_1 \cup L_2 = \{x \mid x \in L_1 \text{ o } x \in L_2\}$$

$$L^0 = \{\epsilon\} \quad L^{n+1} = L \cdot L^n$$

$$L^* = \bigcup_{n \geq 0} L^n$$

$$L^+ = \bigcup_{n \geq 1} L^n$$

# Linguaggio Regolare

6

Def: Un linguaggio  $L \subseteq A^*$  è detto regolare  
sse  $\exists$  una espressione regolare  $r$  tale che  
 $L = \mathcal{L}[r]$

Prop: Ogni linguaggio finito è regolare

Ad es:  $L = \{a, bc\}$      $r = a|bc$

$$\begin{aligned}\mathcal{L}[a|bc] &= \mathcal{L}[a] \cup \mathcal{L}[bc] = \\ &= \{a\} \cup \mathcal{L}[b] \cdot \mathcal{L}[c] = \\ &= \{a\} \cup \{b\} \cdot \{c\} = \\ &= \{a, bc\} = L\end{aligned}$$

(Non serve mai l'operatore  $*$  di ripetizione)

---

Oss: Esistono linguaggi regolari infiniti.

$$\begin{aligned}\mathcal{L}[a^*b] &= \mathcal{L}[a^*] \cdot \mathcal{L}[b] = (\mathcal{L}[a])^* \cdot \{b\} = \\ &= \{a\}^* \cdot \{b\} = \bigcup_{n \geq 0} \{a\}^n \cdot \{b\} = \\ &= \{\epsilon, a, aa, \dots\} \cdot \{b\} = \{a^n b \mid n \geq 0\}\end{aligned}$$

$$\mathcal{L}[a|a^*b] = \mathcal{L}[a] \cup \mathcal{L}[a^*b] = \{a\} \cup \{a^n b \mid n \geq 0\}$$

$$\begin{aligned}\mathcal{L}[(a|b) \cdot b^*] &= \mathcal{L}[a|b] \cdot \mathcal{L}[b^*] = (\mathcal{L}[a] \cup \mathcal{L}[b]) \cdot (\mathcal{L}[b])^* \\ &= \{a, b\} \cdot \{b\}^* = \{ab^n \mid n \geq 0\} \cup \{b^n \mid n \geq 1\}\end{aligned}$$



## Esempi di Espr. Regolari

(6 bis)

$$A = \{0, 1\}$$

$$\bullet 0^* 1 0^*$$

$$L_1 = \{w \in A^* \mid w \text{ contiene un solo } 1\}$$

$$\bullet (0|1)^* 1 (0|1)^*$$

$$L_2 = \{w \in A^* \mid w \text{ contiene almeno un } 1\}$$

$$\bullet (0|1)^* 001 (0|1)^*$$

$$L_3 = \{w \in A^* \mid w \text{ contiene } 001 \text{ come sottostringa}\}$$

$$\bullet 1^* (011^*)^*$$

$$L_4 = \{w \in A^* \mid \text{ogni occorrenza di } 0 \text{ \textit{è} seguita immediatamente da almeno un } 1\}$$

$$\bullet ((0|1)(0|1))^*$$

$$L_5 = \{w \in A^* \mid w \text{ \textit{è} di lunghezza pari}\}$$

$$\bullet (0|1)^* 1$$

$$L_6 = \{w \in A^* \mid w \text{ termina con } 1\}$$

## Altri Operatori Ausiliari

(7)

Ripetizione positiva:  $r^+$  ( $rr^* \text{ o } r^*r$ )

Possibilità:  $r?$  ( $r|\epsilon$ )

Elenco:  $[a_1, \dots, a_n]$  per  $a_1, \dots, a_n \in A$   
(sta per  $a_1|a_2|\dots|a_n$ )

Se gli  $a_i$  sono ordinati in modo che  $a_i < a_{i+1}$   
allora

$[a_1 - a_n]$  (sta per  $a_1|a_2|\dots|a_n$ )

---

### Esempio

• Numeri decimali senza segno

$[0-9]^+ (\epsilon | \cdot [0-9]^+)$

dove l'alfabeto è  $\{0, 1, \dots, 9, \cdot\}$

Alternativamente:  $[0-9]^+ (\cdot [0-9]^+)?$

---

### Definizioni Regolari

Una definizione regolare su alfabeto  $A$  è costituita da una lista di definizioni

$d_1 := r_1$

$d_2 := r_2$

$\vdots$

$d_k := r_k$

dove i vari  $d_i$  sono simboli "nuovi" e ogni  $r_i$  è una espressione regolare sull'alfabeto esteso

$A \cup \{d_1, \dots, d_k\}$



## Esempio

(8)

### (1) Numeri decimali con segno

$\text{numconsegno} := \text{segno} \text{ cifra } (\cdot \text{ cifra})?$

$\text{segno} := - | +$

$\text{cifra} := \text{cifra}^+$

$\text{cifra} := [0-9]$

### (2) Identifier

$\text{identifier} := \text{letter} (\text{letter} | \text{digit})^*$

$\text{letter} := [a-zA-Z]$

$\text{digit} := [0-9]$

Esercizio: Provate a definire un tipo particolare di identificatore che deve soddisfare questi requisiti:

- iniziare con una lettera maiuscola
- contenere almeno una cifra
- terminare col simbolo "!"

# Equivalenza Tra Espr. Regolari

(9)

Def.: Due espressioni regolari  $r$  e  $s$  sono equivalenti se  $L[r] = L[s]$  (cioè denotano lo stesso linguaggio) e lo denotiamo con  $r \simeq s$

Esistono molte leggi per  $\simeq$ . Alcune sono le seguenti:

$$r|s \simeq s|r \quad | \text{ è commutativa}$$

$$r|(s|t) \simeq (r|s)|t \quad | \text{ è associativa}$$

$$r|r \simeq r \quad | \text{ è idempotente}$$

$$r \cdot (s \cdot t) \simeq (r \cdot s) \cdot t \quad \cdot \text{ è associativa}$$

$$\varepsilon \cdot r \simeq r \simeq r \cdot \varepsilon \quad \varepsilon \text{ è l'elemento neutro per } \cdot$$

$$(r^*)^* \simeq r^* \quad * \text{ è idempotente}$$

$$r|(s|t) \simeq rs|rt \quad \cdot \text{ distribuisce a sx su } |$$

$$(r|s)t \simeq rt|st \quad \cdot \text{ distribuisce a dx su } |$$

$$\phi^* \simeq \varepsilon$$

$$(\varepsilon|r)^* \simeq r^*$$

$$r|\phi \simeq r$$

$$(r|s)^* \simeq (r^*|s^*)^*$$

$$r \cdot \phi \simeq \phi \simeq \phi \cdot r$$

⋮



Dimostrare queste ~~opere~~ leggi non è sempre facile, ma in alcuni casi lo è.

$$L[r|s] = L[r] \cup L[s] = L[s] \cup L[r] = L[s|r]$$

$$L[r|r] = L[r] \cup L[r] = L[r]$$

$$L[r\varepsilon] = L[r] \cdot \{\varepsilon\} = L[r]$$

$$\begin{aligned} L[\phi^*] &= (L[\phi])^* = \phi^* = \phi^0 \cup \phi^1 \cup \phi^2 \dots \\ &= \{\varepsilon\} \cup \phi \cup \phi \dots = \{\varepsilon\} = L[\varepsilon] \end{aligned}$$

$$L[r \cdot \phi] = L[r] \cdot L[\phi] = L[r] \cdot \phi = \phi = L[\phi]$$

Le espressioni regolari servono per specificare il pattern di una categoria sintattica, ovvero la forma dei possibili lessemi.

Ma come riconoscere se una certa sequenza in ingresso è un lessema per una certa categoria sintattica?

$\Rightarrow$  Automi (a stati finiti)

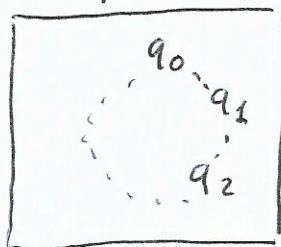
# Automi (a stati) Finiti

(11)

input da leggere

| a | b | a | c | |

| | ← testina  
di lettura



controllo finito

## Caratteristiche:

- memoria finita (data dal numero degli stati:  $q_0, \dots, q_n$ )
- input: stringa da riconoscere
- output: 1 bit (Sì/No)

## Descrizione iniziale

- testina di lettura posizionata sul primo carattere dell'input
- controllo su stato iniziale  $q_0$

## Funzionamento: Ripeti

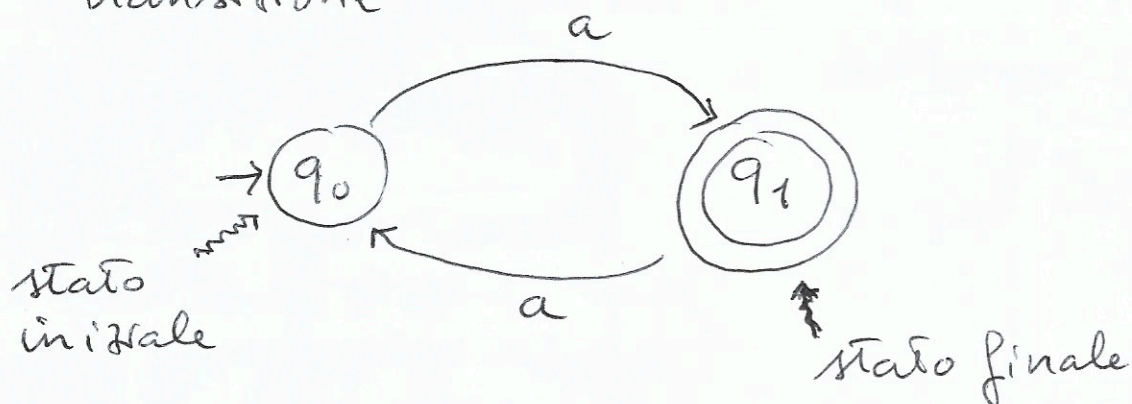
- leggi il carattere in input; in base allo stato in cui si trova decidi:
  - di cambiare di stato
  - e di spostare la testina sull'input successivo fino a che
- ha finito di leggere l'input (e riconosce la stringa se ha raggiunto uno stato finale)
- oppure si è bloccato prima perché per la coppia (stato corrente, input attuale) non era specificato uno stato successivo



# Diagrammi di Transizione

(12)

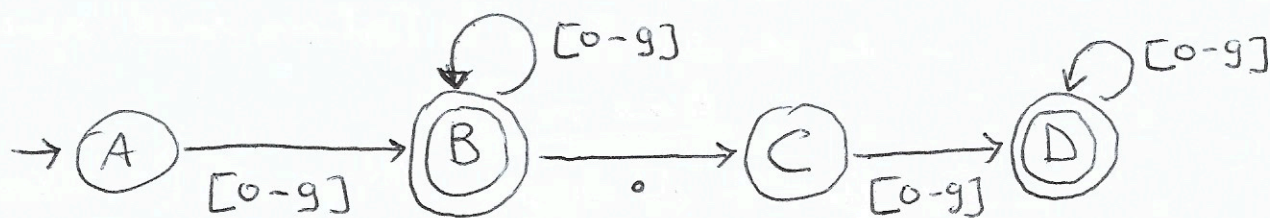
Il funzionamento di un automa finito è ben rappresentato attraverso un diagramma di transizione



Riconoscere una stringa  $w$  significa trovare un cammino etichettato  $w$  sul grafo a partire dallo stato iniziale che finisce su uno stato finale

$$L = \{ a^{2n+1} \mid n \geq 0 \} = \{ a^n \mid n \text{ è dispari} \} \\ = \mathcal{L}[a(aa)^*]$$

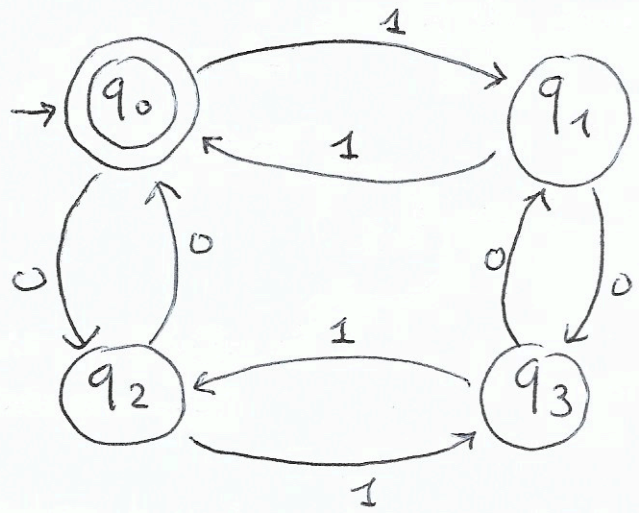
N.B. Se in input ho  $w = ab$ , l'automa si blocca in  $q_1$ , senza essere stata completata la lettura dell'input  $w$



sta a rappresentare 10 diverse transizioni, una per ciascun simbolo  $0, \dots, 9$

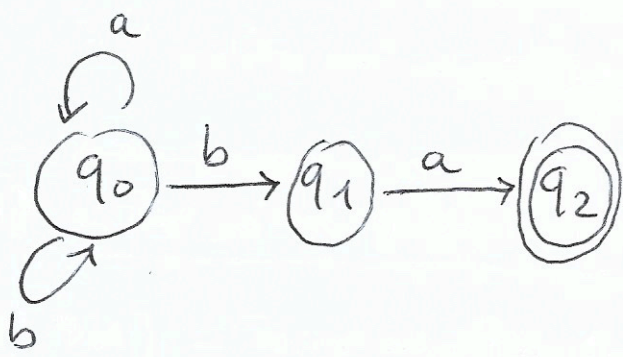
diagramma di transizione per  $[0-9]^+ (\epsilon | . [0-9]^+)$

# Esempi



$L = \{w \in \{0,1\}^* \mid \text{in } w \text{ il numero di 0 e 1 è sempre pari}\}$

N.B. Deterministico:  
per ogni coppia  $(q,s)$ ,  
con  $q \in \{q_0, q_1, q_2, q_3\}$  e  
 $s \in \{0,1\}$ , esiste una e  
una sola mossa possibile

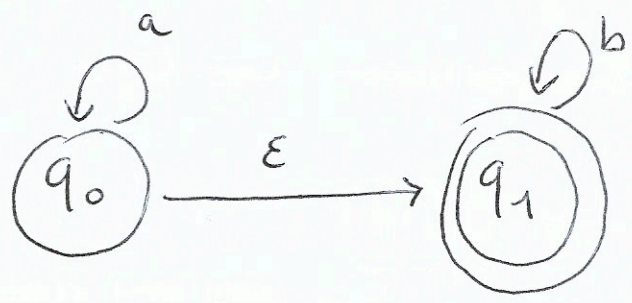


$$L = \mathcal{L}[(ab)^*ba]$$

N.B.: • ~~ba~~  $\in L[M]$  perché  
esiste un cammino da  $q_0$   
a  $q_2$  etichettato  $ba$

• NONDETERMINISTICO

- $(q_0, b)$  offre 2 mosse  
o su  $q_0$  o su  $q_1$
- $(q_1, b)$  non offre mosse
- $(q_2, a/b)$  non offre mosse



$$L[M] = \mathcal{L}[a^*b^*]$$

- nondeterministico:

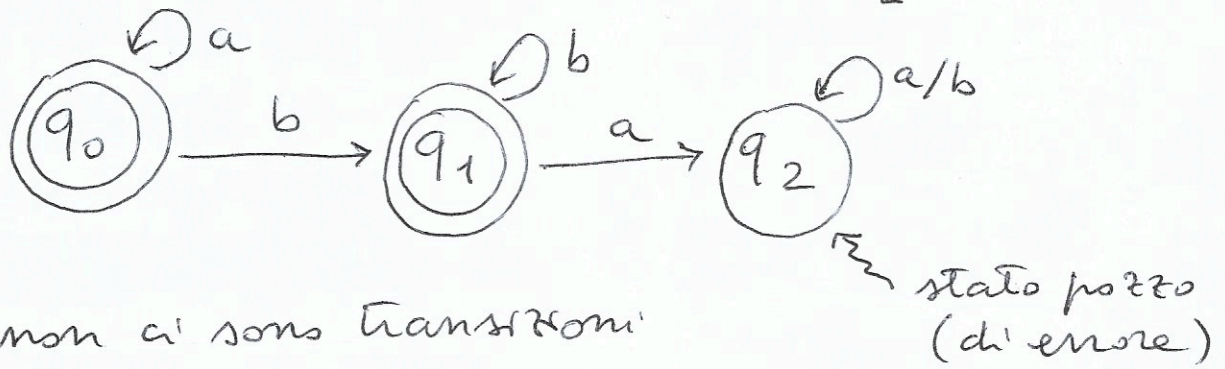
transizione  $\epsilon$



in  $q_0$  può non spostare  
la Testina di lettura  
(ovvero non leggere l'input)  
e spostarsi in  $q_1$



Se vogliamo un automa deterministico (14)  
 per  $L[a^*b^*]$ , possiamo definire



- non ci sono transizioni etichettate  $\epsilon$
  - da ogni stato per ognuno dei due simboli ( $a$  e  $b$ ), esce una e una sola transizione
- ( $\hookrightarrow$ ) questo automa è deterministico

## Automi Finiti Nondeterministici (NFA)

Def: Un automa finito nondeterministico (NFA) è una quintupla  $(\Sigma, Q, \delta, q_0, F)$  dove

- $\Sigma$  è un alfabeto finito di simboli in input
- $Q$  è un insieme finito di stati
- $q_0 \in Q$  è lo stato iniziale
- $F \subseteq Q$  è l'insieme degli stati finali
- $\delta$  è la funzione di transizione con tipo

$$\delta: Q \times (\Sigma \cup \{\epsilon\}) \rightarrow P(Q)$$

$\underbrace{\hspace{1cm}}_{\rightarrow}$  insieme delle parti di  $Q$

$$\left( \delta(q, \sigma) = Q' \subseteq Q \right)$$

## Esempio

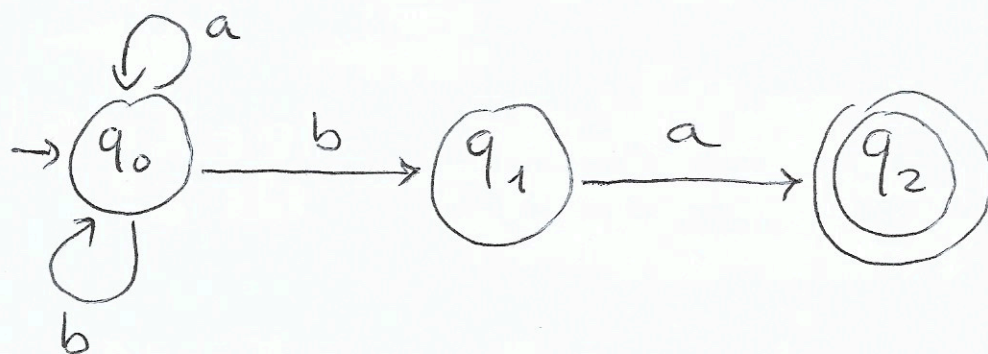
(15)

$\Sigma = \{a, b\}$      $Q = \{q_0, q_1, q_2\}$      $q_0$  iniziale

$F = \{q_2\}$

| $\delta$ | a           | b              | $\epsilon$  |
|----------|-------------|----------------|-------------|
| $q_0$    | $\{q_0\}$   | $\{q_0, q_1\}$ | $\emptyset$ |
| $q_1$    | $\{q_2\}$   | $\emptyset$    | $\emptyset$ |
| $q_2$    | $\emptyset$ | $\emptyset$    | $\emptyset$ |

Come rappresentare questo NFA con un diagramma di transizioni?



Esercizio • Disegnate il diagramma di transizione per  $M = (\Sigma, Q, \delta, q_0, F)$  dove

$\Sigma = \{0, 1\}$      $Q = \{q_0, q_1, q_2\}$      $q_0$  iniziale     $F = \{q_1\}$

| $\delta$ | 0              | 1              | $\epsilon$  |
|----------|----------------|----------------|-------------|
| $q_0$    | $\emptyset$    | $\{q_0, q_2\}$ | $\{q_1\}$   |
| $q_1$    | $\{q_1\}$      | $\emptyset$    | $\{q_0\}$   |
| $q_2$    | $\{q_2, q_0\}$ | $\emptyset$    | $\emptyset$ |

- Ricavare dai diagrammi di transizione a pg 14 la definizione di NFA come quintupla  $(\Sigma, Q, \delta, q_0, F)$ .



Def (informale) Un NFA  $N = (\Sigma, Q, \delta, q_0, F)$  accetta  $w = a_1 \dots a_n$  se nel diagramma di transizione esiste un cammino da  $q_0$  ad uno stato in  $F$  nel quale la stringa che si ottiene concatenando le etichette degli archi percorsi è esattamente  $w$ .

Più formalmente:

• Descrizione istantanea:  $(q, w)$

$\uparrow$                        $\nwarrow$   
 stato                  input da  
 corrente              leggere

• Mossa                       $q' \in \delta(q, \sigma)$                        $\sigma \in \Sigma \cup \{\epsilon\}$   
 $(q, \sigma w) \vdash_N (q', w)$                        $w \in \Sigma^*$

• Cammino (chiusura riflessiva e transitiva di  $\vdash_N$ )

$(q, w) \vdash_N^* (q', w')$      $(q', w') \vdash_N (q'', w'')$   


---

 $(q, w) \vdash_N^* (q, w)$                        $(q, w) \vdash_N^* (q'', w'')$

• Accettazione/Riconoscimento

$w \in L[N]$  se  $\exists q \in F. (q_0, w) \vdash_N^* (q, \epsilon)$

- Il linguaggio accettato da  $N$ , indicato con  $L[N]$ , è

$L[N] = \{w \in \Sigma^* \mid \exists q \in F. (q_0, w) \vdash_N^* (q, \epsilon)\}$

- Due NFA  $N_1$  e  $N_2$  si dicono equivalenti se accettano lo stesso lang., cioè se  $L[N_1] = L[N_2]$

Costruire un NFA per i linguaggi

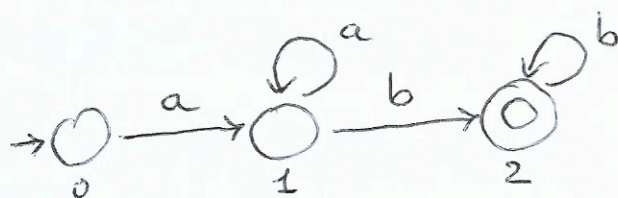
$$L_1 = \{ a^m b^m \mid m, m \geq 1 \}$$

$$L_2 = \{ a^{3k+2} \mid k \geq 0 \}$$

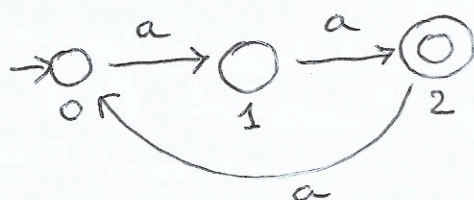
$$L_3 = \{ w \in (a|b)^* \mid w \text{ contiene la sottosequenza } aba \}$$

$$L_4 = \{ w \in (a|b)^* \mid w \text{ contiene esattamente 3 "a"} \}$$

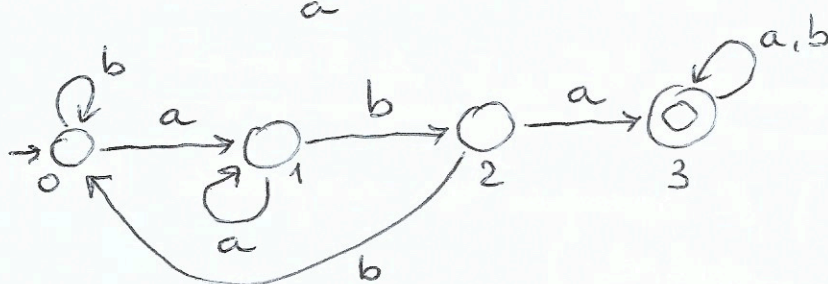
Solutions



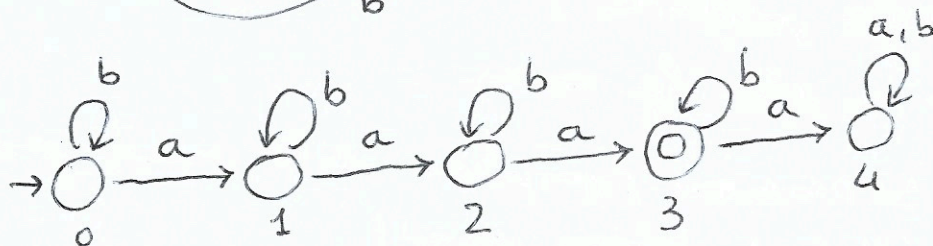
$$L[N_1] = L_1$$



$$L[N_2] = L_2$$



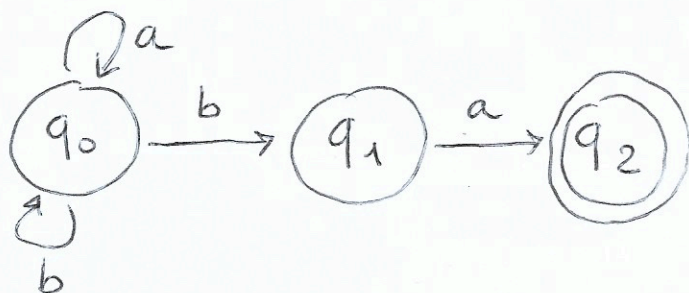
$$L[N_3] = L_3$$



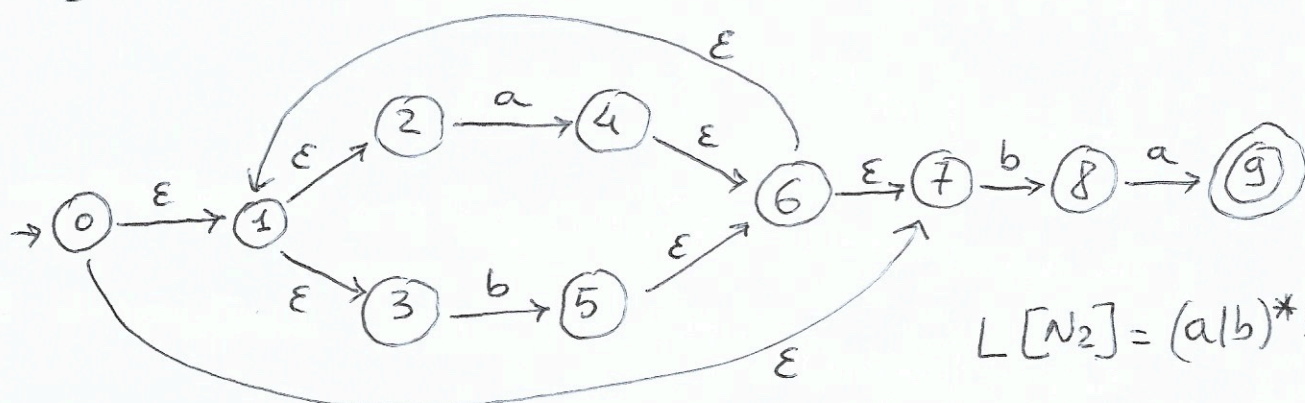
$$L[N_4] = L_4$$

N.B. Per un linguaggio  $L$ , se esiste un NFA  $N$  tale che  $L = L[N]$ , allora  $\exists$  infiniti  $N_1, N_2, \dots, N_k, \dots$  tali che  $L = L[N_i]$  per  $i = 1, 2, \dots$





$$L[N_1] = L[(a|b)^*ba]$$



$$L[N_2] = (a|b)^*ba$$

e ne vedremo altri sempre per questo linguaggio!

NFA: - sono "comodi": facile costruirli, ma  
 - inefficienti: accettare  $w$  significa cercare un cammino su un grafo  
 "nondeterministico"  $\Rightarrow$  tante potenziali strade alternative (possibile failure  $\Rightarrow$  backtracking)

DFA: Deterministic Finite Automata

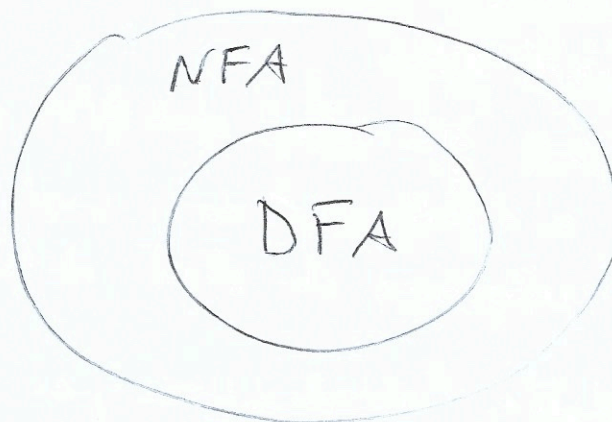
- $\delta(q, c)$  è sempre un singolo (solo una mossa possibile)
  - non ci sono le mosse  $\epsilon$
- $\Rightarrow$
- scansione completa dell'input garantita (non si blocca mai)
  - in un tempo  $O(n)$  - dove  $n$  è la lunghezza di  $w$  - sappiamo se  $w$  è accettata o meno
  - ma più difficili da definire

Def Un automa finito deterministico (DFA) è una quintupla  $(\Sigma, Q, \delta, q_0, F)$ , dove  $\Sigma, Q, q_0$  e  $F$  sono definiti come per un NFA, mentre la funzione di transizione  $\delta$  ha tipo  $\delta: Q \times \Sigma \rightarrow Q$

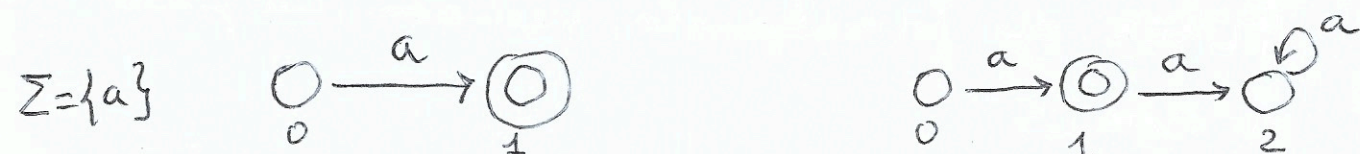
$$(\delta(q, \sigma) = q')$$

Osservazione: Un DFA è un particolare tipo di NFA tale che:

- $\forall q \in Q \quad \delta(q, \epsilon) = \emptyset$  (non ci sono transizioni  $\epsilon$ )
- $\forall \sigma \in \Sigma, \forall q \in Q \exists q' \in Q. \delta(q, \sigma) = \{q'\}$   
(l'insieme delle mosse possibili è sempre un)   
singoletto

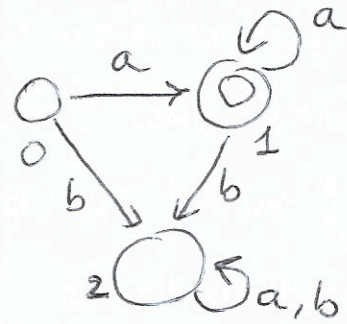
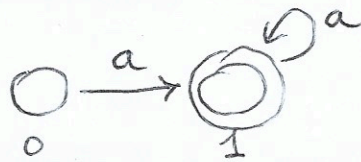


NFA o DFA ?





$\Sigma = \{a, b\}$



Vogliamo ora dimostrare che i DFA sono tanto espressivi quanto gli NFA, sebbene siano un sottoinsieme proprio degli NFA.

Prop: Per ogni NFA, è possibile costruire un DFA ad esso equivalente!

Come fare? Idea: "seguire" contemporaneamente tutti i possibili cammini alternativi dell'NFA

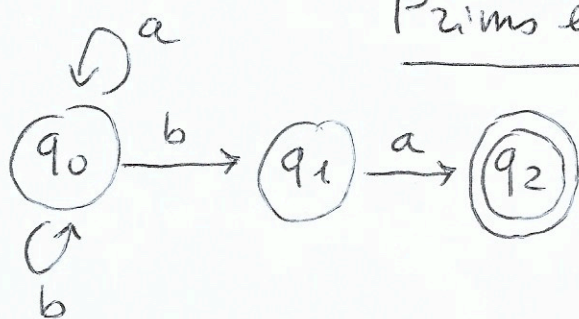
$\Rightarrow$  gli stati del DFA che andiamo a costruire sono costituiti da insiemi di stati dell'NFA

Tecnica detta "Costruzione per sottoinsiemi"  
(Subset Construction)

Prima di formalizzarla, vediamo un paio d'esempi.

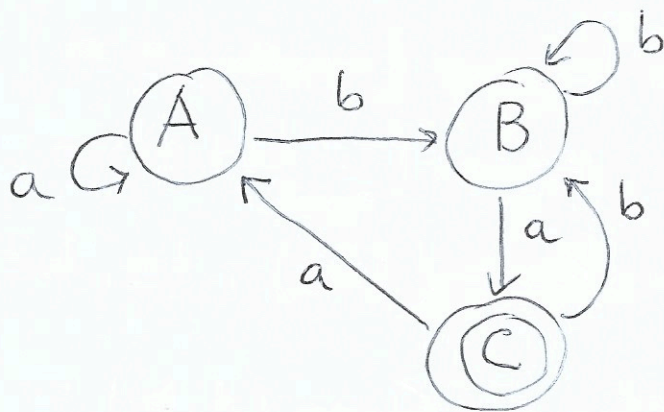
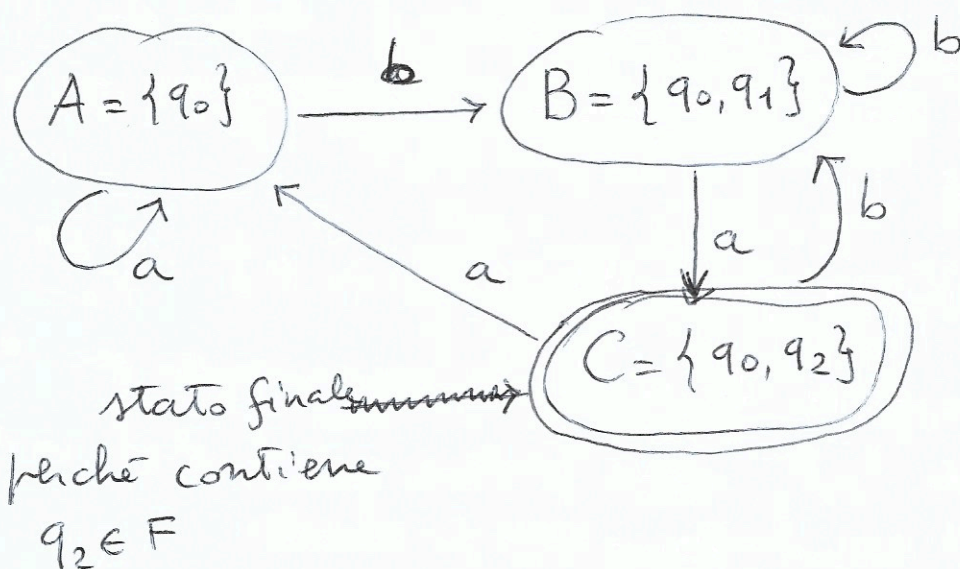
## Primo esempio semplice

(21)



NFA che vogliamo trasformare in un equivalente DFA

Stato iniziale del DFA:  $\{q_0\} = A$

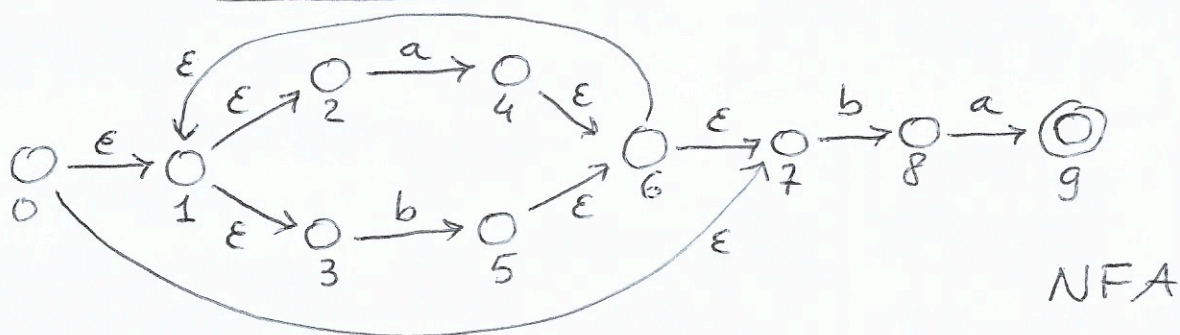


questo è un DFA del tutto equivalente

- Passando dallo stato  $A = \{q_0\}$  allo stato  $B = \{q_0, q_1\}$ , ho seguito contemporaneamente tutte e due le transizioni etichettate  $b$
- Passando dallo stato  $B = \{q_0, q_1\}$  a  $C = \{q_0, q_2\}$ , ho seguito sia la transizione  $q_0 \xrightarrow{a} q_0$ , sia la transizione  $q_1 \xrightarrow{a} q_2$

$$\begin{aligned} \text{mossa}(B, a) &= \bigcup_{q \in B} \delta(q, a) = \delta(q_0, a) \cup \delta(q_1, a) \\ &= \{q_0, q_2\} = C \end{aligned}$$





Lo stato iniziale del DFA è dato da 0, più tutti gli stati che possono raggiungere da 0 con mosse  $\epsilon$

$$\epsilon\text{-closure}(0) = \{0, 1, 2, 3, 7\} \quad \textcircled{A}$$

Per calcolare lo stato che raggiungo da A leggendo "a", devo vedere quali stati in A possono fare "a" e poi farne la  $\epsilon$ -closure

$$\begin{aligned} \Delta(A, a) &= \epsilon\text{-closure}(\text{mossa}(A, a)) = \\ &= \epsilon\text{-closure}\left(\bigcup_{q \in A} \delta(q, a)\right) = \\ &= \epsilon\text{-closure}(\{4\}) = \{4, 6, 7, 1, 2, 3\} = B \end{aligned}$$

$$\begin{aligned} \Delta(A, b) &= \epsilon\text{-closure}(\text{mossa}(A, b)) = \\ &= \epsilon\text{-closure}\left(\bigcup_{q \in A} \delta(q, b)\right) = \\ &= \epsilon\text{-closure}(\{5, 8\}) = \{1, 2, 3, 5, 6, 7, 8\} = C \end{aligned}$$

$$\Delta(B, a) = \epsilon\text{-closure}(\{4\}) = B$$

$$\Delta(B, b) = \epsilon\text{-closure}(\{5, 8\}) = C$$

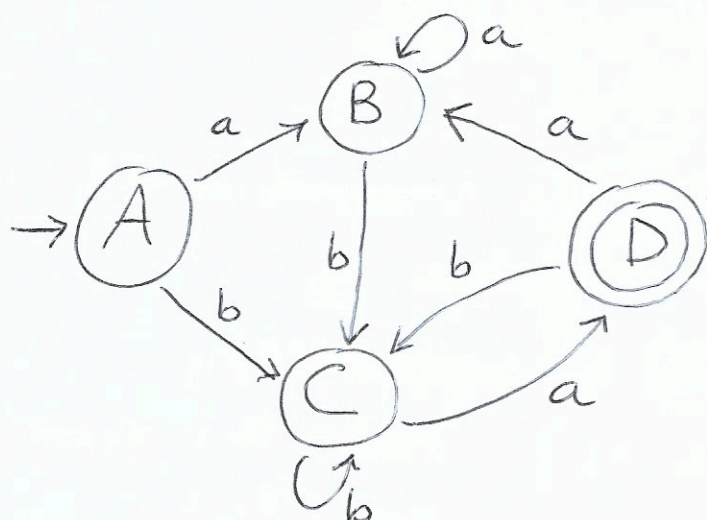
$$\Delta(C, a) = \epsilon\text{-closure}(\{4, 9\}) = \{4, 9, 1, 2, 3, 6, 7\} = D$$

$$\Delta(C, b) = \epsilon\text{-closure}(\{5, 8\}) = C$$

$$\Delta(D, a) = \varepsilon\text{-closure}(\{4\}) = B$$

(23)

$$\Delta(D, b) = \varepsilon\text{-closure}(\{5, 8\}) = C$$



DFA ottenuto  
per costruzione  
per sottoinsiemi

### $\varepsilon$ -closure e mosse

Sia  $q$  uno stato di un NFA. La  $\varepsilon$ -closure di  $q$  è l'insieme degli stati raggiungibili da  $q$  solo con mosse  $\varepsilon$ .

$$\frac{}{\{q\} \subseteq \varepsilon\text{-closure}(q)} \quad \frac{p \in \varepsilon\text{-closure}(q)}{\delta(p, \varepsilon) \subseteq \varepsilon\text{-closure}(q)}$$

Sia  $P$  un insieme di stati di un NFA.

$$\varepsilon\text{-closure}(P) = \bigcup_{p \in P} \varepsilon\text{-closure}(p)$$

$$\text{mosse} : \mathcal{P}(Q) \times \Sigma \rightarrow \mathcal{P}(Q)$$

$$\text{mosse}(P, a) = \bigcup_{p \in P} \delta(p, a)$$

$$\Delta(A, b) = \varepsilon\text{-closure}(\text{mosse}(A, b))$$

↑  
funzione di transizione del DFA



## Algoritmo per calcolare la $\epsilon$ -closure(P)

(24)

Inizializzare  $T = P$ ;

Inizializzare  $\epsilon\text{-closure}(P) = P$ ;

while  $T \neq \emptyset$  do {

    "scegli un  $r \in T$  e rimuovilo da  $T$ "

for each  $s \in \delta(r, \epsilon)$  do

        if  $s \notin \epsilon\text{-closure}(P)$  {

            add  $s$  to  $\epsilon\text{-closure}(P)$ ;

            add  $s$  to  $T$ ;

        }

    }

---

Oss: Usando  $\epsilon$ -closure, si può definire il lang. riconosciuto da un NFA in modo elegante:

$$\hat{\delta}: Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$$

$$\hat{\delta}(q, \epsilon) = \epsilon\text{-closure}(q)$$

$$\hat{\delta}(q, xa) = \epsilon\text{-closure}(P)$$

$$\text{dove } P = \{p \in Q \mid \exists r \in \hat{\delta}(q, x) \text{ e } p \in \delta(r, a)\}$$

$w \in L[N] \text{ se } \exists p \in F \text{ tale che } p \in \hat{\delta}(q_0, w)$

Equivalentemente

$$\begin{aligned} \hat{\delta}(q, xa) &= \epsilon\text{-closure}(\text{move}(\hat{\delta}(q, x), a)) \\ &= \epsilon\text{-closure}\left(\bigcup_{p \in \hat{\delta}(q, x)} \delta(p, a)\right) \end{aligned}$$

Dato un NFA  $N = (\Sigma, Q, \delta, q_0, F)$ :

- Inizializza  $S = \varepsilon\text{-closure}(q_0)$ ; //  $S$  stato iniziale del DFA
- Inizializza  $T = \{S\}$ ; //  $T$  è l'insieme degli stati del DFA  
//  $S$  non è marcato all'inizio
- Finché c'è un  $P \in T$  non marcato {
  - marca  $P$ ;
  - for each  $a \in \Sigma$  {
    - $R = \varepsilon\text{-closure}(\text{mossa}(P, a))$ ;
    - if  $R \notin T$  {
      - add  $R$  to  $T$ ; //  $R$  non ha marca
  - definisci  $\Delta(P, a) = R$ ;}

---

Definiamo il DFA  $M_N = (\Sigma, T, \Delta, \varepsilon\text{-closure}(q_0), F)$   
dove  $R \in F$  se  $\exists q \in R$  con  $q \in F$ .

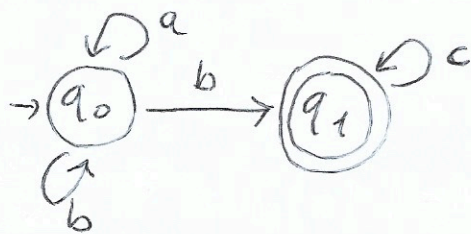
---

- Nel caso pessimo  $T = P(Q)$ , cioè il DFA  $M_N$  costruito a partire dall'NFA  $N$ , ha un numero di stati pari a  $2^m$ , dove  $m = |Q|$ .  
↑ esponenziale
- Di solito  $T$  è molto più piccolo di  $P(Q)$

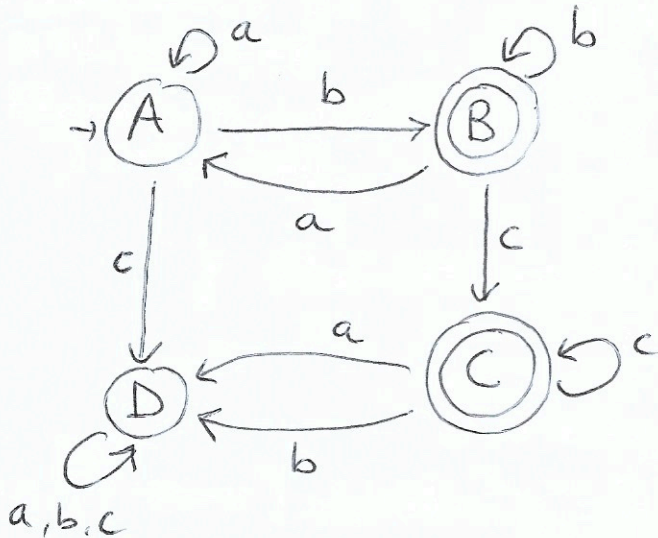


# Esempio di caso pessimo

(26)



NFA  $N$  con 2 stati.



$$A = \{q_0\}$$

$$B = \{q_0, q_1\}$$

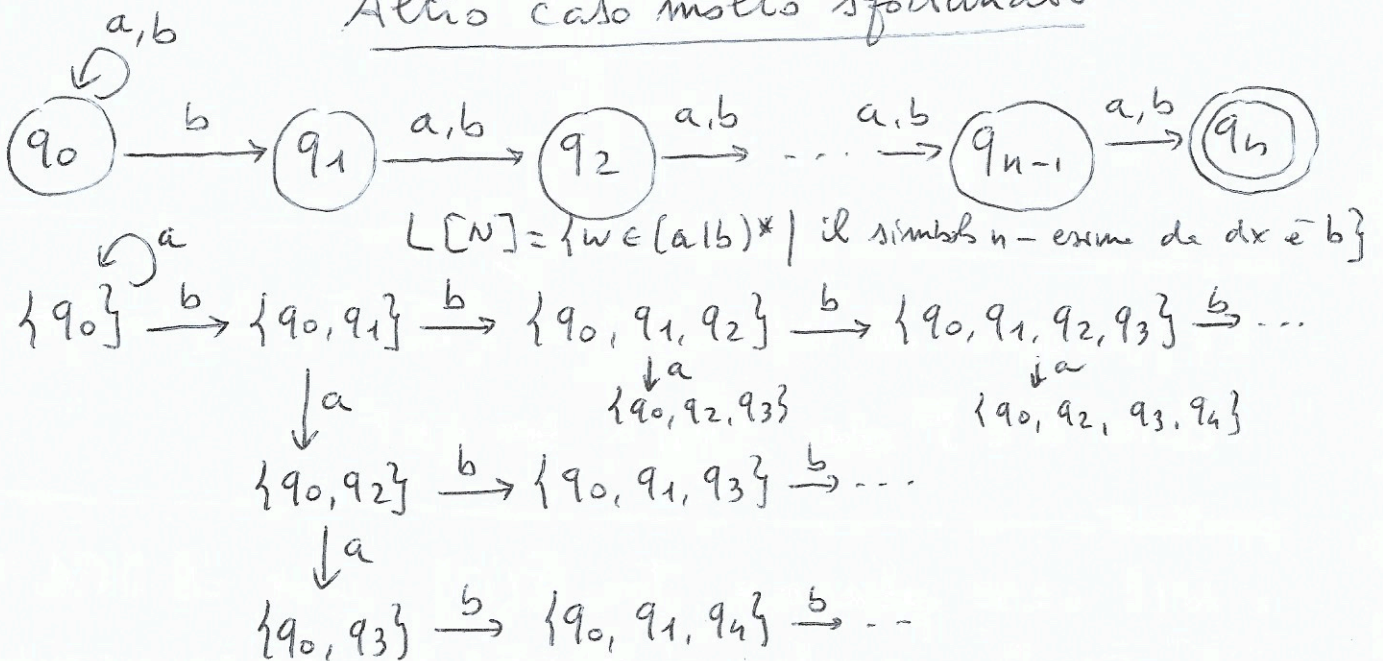
$$C = \{q_1\}$$

$$D = \emptyset$$

DFA  $M_N$  con  $2^2 = 4$  stati.

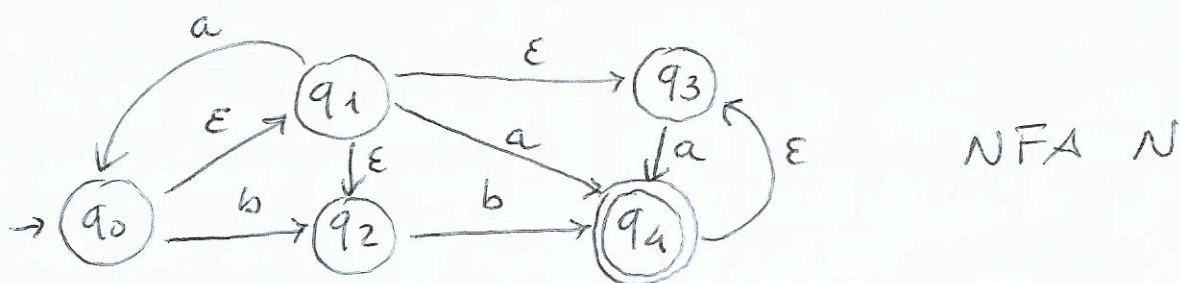
Il DFA  $M_N$  può essere "esponenzialmente" più grande dell'NFA  $N$

## Altro caso molto sfortunato



Anche in questo caso, gli stati del DFA sono  $2^n$  mentre gli stati dell'NFA sono  $n+1$ .

Provate a disegnare il DFA per  $q_0 \xrightarrow{a,b} q_1 \xrightarrow{a,b} q_2$ . Vedrete che avrà  $2^2$  stati.



$$A = \epsilon\text{-closure}(q_0) = \{q_0, q_1, q_2, q_3\}$$

$$\begin{aligned} \Delta(A, a) &= \epsilon\text{-closure}(\text{move}(A, a)) = \epsilon\text{-closure}(\{q_0, q_4\}) = \\ &= \{q_0, q_1, q_2, q_3, q_4\} = B \end{aligned}$$

$$\begin{aligned} \Delta(A, b) &= \epsilon\text{-closure}(\text{move}(A, b)) = \epsilon\text{-closure}(\{q_2, q_4\}) = \\ &= \{q_2, q_3, q_4\} = C \end{aligned}$$

$$\Delta(B, a) = \epsilon\text{-closure}(\text{move}(B, a)) = \epsilon\text{-closure}(\{q_0, q_4\}) = B$$

$$\Delta(B, b) = \epsilon\text{-closure}(\text{move}(B, b)) = \epsilon\text{-closure}(\{q_2, q_4\}) = C$$

$$\begin{aligned} \Delta(C, a) &= \epsilon\text{-closure}(\text{move}(C, a)) = \epsilon\text{-closure}(\{q_4\}) = \\ &= \{q_3, q_4\} = D \end{aligned}$$

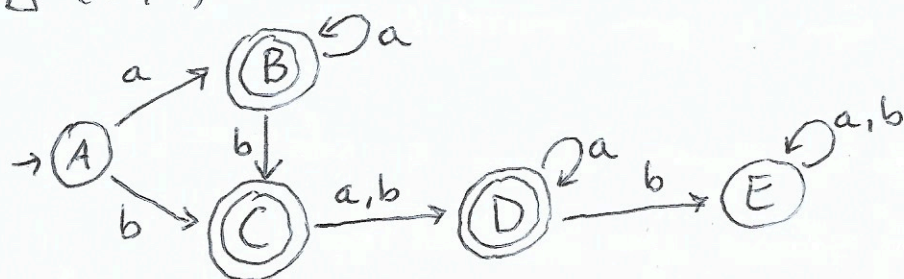
$$\Delta(C, b) = \epsilon\text{-closure}(\text{move}(C, b)) = \epsilon\text{-closure}(\{q_4\}) = D$$

$$\Delta(D, a) = \epsilon\text{-closure}(\text{move}(D, a)) = \epsilon\text{-closure}(\{q_4\}) = D$$

$$\Delta(D, b) = \epsilon\text{-closure}(\text{move}(D, b)) = \epsilon\text{-closure}(\emptyset) = \emptyset = E$$

$$\Delta(E, a) = \epsilon\text{-closure}(\text{move}(E, a)) = \epsilon\text{-closure}(\emptyset) = \emptyset = E$$

$$\Delta(E, b) = \epsilon\text{-closure}(\text{move}(E, b)) = \epsilon\text{-closure}(\emptyset) = \emptyset = E$$





## Equivalenza NFA - DFA

(28)

Teorema Sia  $N = (\Sigma, Q, \delta, q_0, F)$  un NFA e sia  $M_N$  l'automa ottenuto con la costruzione per sottoinsiemi. Allora  $M_N$  è un DFA e si ha che

$$L[N] = L[M_N]$$

Corollario La classe dei linguaggi riconosciuti dagli NFA coincide con la classe dei lang. riconosciuti dai DFA

---

### Dimostrazione del Teorema:

Sia  $N = (\Sigma, Q, \delta, q_0, F)$  un NFA e sia  $M_N = (\Sigma, T, \Delta, \epsilon\text{-closure}(q_0), F)$  l'automa ottenuto con l'algoritmo.

(1)  $M_N$  è deterministico! Infatti  $\Delta(A, a)$  è definita per ogni coppia  $(A, a)$  con  $A \in T$  e  $a \in \Sigma$  in modo univoco, e il risultato di  $\Delta(A, a)$  è un elemento di  $T$ .

(2) Quindi dobbiamo "solo" dimostrare che  $L[N] = L[M_N]$

Oss: Per un DFA,  $\epsilon\text{-closure}(R) = R$  con  $R \in T$ , perché non ci sono mosse  $\epsilon$

Notazione Chiamiamo  $i_M = \epsilon\text{-closure}(q_0)$  lo stato iniziale di  $M_N$

Vogliamo dimostrare che  $\forall w \in \Sigma^*$

(29)

$$\hat{\delta}(q_0, w) = \hat{\Delta}(i_H, w)$$

(Per def.  $\hat{\delta}$   
vedi pg. 24)

per induzione sulla lunghezza di  $w$ .

Caso base :  $|w| = 0$  cioè  $w = \epsilon$

$$\hat{\delta}(q_0, \epsilon) = \epsilon\text{-closure}(q_0)$$

$$\hat{\Delta}(i_H, \epsilon) = \epsilon\text{-closure}(i_H) = i_H = \epsilon\text{-closure}(q_0)$$

Passo induttivo  $w = xa$  con  $a \in \Sigma$  e  $x \in \Sigma^*$  ok

Per ipotesi induttiva, sappiamo che

$$\hat{\delta}(q_0, x) = \hat{\Delta}(i_H, x) = \{p_1, \dots, p_k\}$$

Per definizione di  $\hat{\delta}$ ,

$$\hat{\delta}(q_0, xa) = \epsilon\text{-closure}\left(\bigcup_{i=1}^k \delta(p_i, a)\right)$$

Similmente

$$\hat{\Delta}(i_H, xa) = \Delta(\{p_1, \dots, p_k\}, a)$$

In base all'algoritmo, la definizione di  $\Delta$  ci dice che

$$\begin{aligned} \Delta(\{p_1, \dots, p_k\}, a) &= \epsilon\text{-closure}(\text{mossa}(\{p_1, \dots, p_k\}, a)) = \\ &= \epsilon\text{-closure}\left(\bigcup_{i=1}^k \delta(p_i, a)\right) \\ &= \hat{\delta}(q_0, xa) \end{aligned}$$

ok



Infine, abbiamo che

(30)

$w \in L[N]$  se  $\exists p \in \hat{\delta}(q_0, w)$  con  $p \in F$

se  $\exists p \in \hat{\Delta}(i_M, w)$  con  $p \in F$

se  $\hat{\Delta}(i_M, w) \in \mathcal{F}$

se  $w \in L[M_N] \quad \forall w \in \Sigma^*$

Quindi  $L[N] = L[M_N]$  c.v.d.

---